

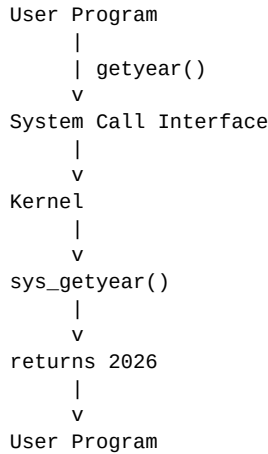
Adding System Calls in xv6

Complete exam + practical guide for xv6-public

This guide combines the complete procedure, file-by-file changes, system-call flow, and multiple solved examples.

1. What is a System Call?

A system call is a mechanism through which a user program requests a service from the operating system kernel. In xv6, a user-level function such as **getpid()** eventually enters the kernel, where a corresponding kernel function such as **sys_getpid()** performs the work.

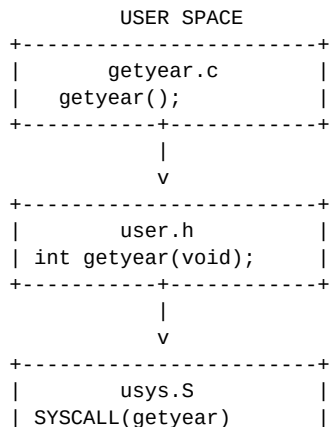


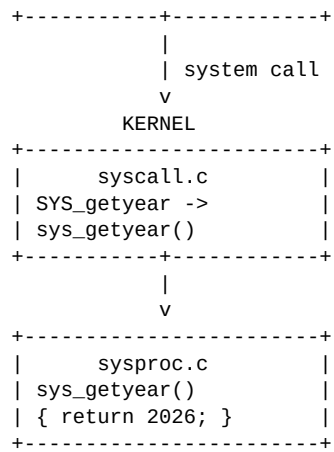
2. Files Involved

For a typical custom system call in xv6-public, the main files are:

File	What you add	Purpose
syscall.h	SYS_name	Give the syscall a number
syscall.c	extern + syscall table entry	Connect number to kernel function
sysproc.c	sys_name()	Implement process/general syscall logic
user.h	name(...)	Declare the user-level function
usys.S	SYSCALL(name)	Create user-to-kernel entry
Makefile	_name	Build and include the test program
name.c	main()	Test the syscall

3. Complete Flow of a System Call





The key idea is: the user function is connected to a system-call number, and that number is mapped by the kernel to the actual sys_* implementation.

4. Detailed Example: getyear()

Question: Add a system call `getyear()` that returns 2026.

Step 1 — `syscall.h`: give the syscall a number.

```
#define SYS_getyear 22
```

Use the next free number in your own copy of `xv6`; do not blindly use 22 if it is already occupied.

Step 2 — `sysproc.c`: implement the kernel function.

```
int
sys_getyear(void)
{
    return 2026;
}
```

Step 3 — `syscall.c`: declare the function.

```
extern int sys_getyear(void);
```

Step 4 — `syscall.c`: add it to the syscall table.

```
[SYS_getyear] sys_getyear,
```

This creates the mapping: `SYS_getyear` → `sys_getyear()`.

Step 5 — `user.h`: declare the user-level function.

```
int getyear(void);
```

Step 6 — `usys.S`: add the user-to-kernel entry.

```
SYSCALL(getyear)
```

Step 7 — `getyear.c`: create a user program.

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
    printf(1, "Year = %d\n", getyear());
    exit();
}
```

Step 8 — `Makefile`: add the program to `UPROGS`.

```
_getyear\\
```

Run:

```
make clean
make
make qemu
# or, depending on your setup:
make qemu-nox
```

Inside `xv6`:

```
$ getyear
Year = 2026
```

5. Why Each File is Needed

`syscall.h`: assigns an ID to the system call.

syscall.c: tells the kernel which function corresponds to that ID.

sysproc.c: contains the actual implementation for process/general system calls.

user.h: lets user programs compile calls such as `getyear()`.

usys.S: creates the assembly-level user-side system-call entry.

Makefile: makes sure the test program is compiled and included in the xv6 filesystem.

6. Multiple System Call Examples

Example 1 — getyear(): no arguments

```
int
sys_getyear(void)
{
    return 2026;
}
```

Useful for learning the basic syscall mechanism.

Example 2 — getppid(): process information

Returns the parent process ID.

```
int
sys_getppid(void)
{
    return myproc()->parent->pid;
}
```

User declaration:

```
int getppid(void);
```

Test:

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
    printf(1, "My PID = %d\n", getpid());
    printf(1, "Parent PID = %d\n", getppid());
    exit();
}
```

The exact PIDs vary. The important concept is **myproc()** → current process → parent → parent PID.

Example 3 — countproc(): count active processes

```
int
sys_countproc(void)
{
    struct proc *p;
    int count = 0;

    acquire(&ptable.lock);

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
    {
        if(p->state != UNUSED)
            count++;
    }

    release(&ptable.lock);

    return count;
}
```

The process table is protected with ptable.lock because it can be accessed or modified concurrently.

Example 4 — getparentname(): string/process information

```
int
sys_getparentname(void)
{
    cprintf("Parent process: %s\n", myproc()->parent->name);
    return 0;
}
```

```
}
```

This demonstrates accessing process names. For a real user-space return of a string, xv6 must copy data safely between user and kernel address spaces.

7. System Calls with Arguments

Example 5 — add(a,b)

Question: Add a syscall that returns the sum of two integers.

syscall.h:

```
#define SYS_add 25
```

sysproc.c:

```
int
sys_add(void)
{
    int a, b;

    if(argint(0, &a) < 0)
        return -1;

    if(argint(1, &b) < 0)
        return -1;

    return a + b;
}
```

user.h:

```
int add(int, int);
```

usys.S:

```
SYSCALL(add)
```

Test program:

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
    int result;

    result = add(10, 20);

    printf(1, "Result = %d\n", result);
    exit();
}
```

Output:

```
Result = 30
```

Example 6 — square(n)

Question: Return the square of an integer.

```
int
sys_square(void)
{
    int n;

    if(argint(0, &n) < 0)
        return -1;

    return n * n;
}
```

User declaration:

```
int square(int);
```

Test:

```
printf(1, "Square = %d\n", square(5));
```

Output: **Square = 25**

Example 7 — isprime(n)

A logic-based syscall returning 1 for prime and 0 otherwise.

```
int
sys_isprime(void)
{
    int n;
    int i;

    if(argint(0, &n) < 0)
        return -1;

    if(n < 2)
        return 0;

    for(i = 2; i * i <= n; i++)
    {
        if(n % i == 0)
            return 0;
    }

    return 1;
}
```

8. Process Information Example

Example 8 — getprocessinfo()

Display information about the current process.

```
int
sys_getprocessinfo(void)
{
    struct proc *p;

    p = myproc();

    cprintf("PID : %d\n", p->pid);
    cprintf("Name : %s\n", p->name);
    cprintf("State: %d\n", p->state);

    return 0;
}
```

Test program:

```
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
    getprocessinfo();
    exit();
}
```

The state number depends on the xv6 process-state enum.

9. String Argument Example — printmsg()

This example teaches **argstr()**.

```
int
sys_printmsg(void)
{
    char *msg;

    if(argstr(0, &msg) < 0)
        return -1;

    cprintf("Message: %s\n", msg);

    return 0;
}
```

user.h:

```
int printmsg(char*);
```

Test:

```
int
main(void)
{
    printmsg("Hello from user space");
    exit();
}
```

Output:

```
Message: Hello from user space
```

10. File-Related System Call

Not every syscall belongs in `sysproc.c`. File-related system calls are generally implemented in **`sysfile.c`**.

Example 9 — `getfilesize(fd)`

Return the size of an opened file. This is more advanced because it uses a file descriptor and xv6 file/inode structures.

```
int
sys_getfilesize(void)
{
    struct file *f;
    int fd;

    if(argint(0, &fd) < 0)
        return -1;

    if(argfd(fd, 0, &f) < 0)
        return -1;

    return f->ip->size;
}
```

Important concepts:

```
argint()
|
v
file descriptor
|
v
argfd()
|
v
struct file
|
v
inode
|
v
size
```

11. Which File Should I Modify?

Requirement	Typical file
Assign syscall number	<code>syscall.h</code>
Process/general syscall implementation	<code>sysproc.c</code>
File syscall implementation	<code>sysfile.c</code>
Register syscall function	<code>syscall.c</code>
Map syscall number to function	<code>syscall.c</code>
Declare user function	<code>user.h</code>
User-to-kernel assembly entry	<code>usys.S</code>
Compile user test program	<code>Makefile</code>

12. Important xv6 Functions to Know

`argint()` — retrieve an integer argument supplied by the user.

`argstr()` — retrieve a string argument.

`argfd()` — retrieve/validate a file descriptor and associated file structure.

myproc() — obtain the current process structure.

acquire()/release() — protect shared kernel data structures with locks.

cprintf() — print from kernel code.

13. Complete Exam Skeleton

Memorize this structure rather than memorizing every individual example.

1. syscall.h — give the syscall a number:

```
#define SYS_name NUMBER
```

2. Kernel implementation — usually sysproc.c for process/general work:

```
int
sys_name(void)
{
    return ...;
}
```

For an integer argument:

```
int
sys_name(void)
{
    int x;

    if(argint(0, &x) < 0)
        return -1;

    ...
}
```

For a string:

```
int
sys_name(void)
{
    char *str;

    if(argstr(0, &str) < 0)
        return -1;

    ...
}
```

3. syscall.c — declaration:

```
extern int sys_name(void);
```

4. syscall.c — syscall table:

```
[SYS_name] sys_name,
```

5. user.h — user declaration:

```
int name(...);
```

6. usys.S — user-side entry:

```
SYSCALL(name)
```

7. Makefile — add test program:

```
_name\\
```

14. Quick Revision: The 6 Main Steps

Step 1: Number
syscall.h
#define SYS_name NUMBER

Step 2: Implement
sysproc.c / sysfile.c
sys_name()

Step 3: Declare + Map
syscall.c
extern int sys_name(void);
[SYS_name] sys_name,

Step 4: User declaration
user.h
int name(...);

Step 5: Assembly bridge
usys.S
SYSCALL(name)

Step 6: Test program
name.c
Makefile
_name

Memory trick:

Number → Implementation → Mapping → User declaration → Assembly bridge → Makefile

15. Five Examples to Practice for an Exam

Level	Practice syscall	What you learn
1	getyear()	Basic syscall with no arguments
2	square(n)	argint()
3	add(a,b)	Multiple arguments
4	getppid()	myproc() and process structures
5	getfilesize(fd)	argfd(), file/inode structures

If you can implement these five without looking at the solution, you understand most of the mechanics involved in adding basic xv6 system calls.

16. Final Checklist Before Running

- [] Pick an unused SYS_ number
- [] Add #define in syscall.h
- [] Write sys_name() in the correct kernel file
- [] Add extern declaration in syscall.c
- [] Add [SYS_name] sys_name to syscall table
- [] Add function declaration to user.h
- [] Add SYSCALL(name) to usys.S
- [] Create name.c test program
- [] Add _name to Makefile
- [] make clean
- [] make
- [] make qemu / make qemu-nox
- [] Run \$ name

Note: Exact source layout and existing syscall numbers can vary slightly between xv6 variants. The guide is written for the xv6-public style you have been using.